1. Why do we even need RAG?

LLMs like GPT have two types of “knowledge”:

(A) Prior Knowledge — learned during training

Example: *“Shakespeare’s wife was Anne Hathaway.”*

(B) Contextual Knowledge — new info provided in the prompt

Example:
“Abraham Shakespeare was a lottery winner in 2006.”

Now your teacher gives the test:

> **Only use the new context. Who was Shakespeare’s wife?**

The correct answer should be:

> **“I don’t know — the context doesn’t contain that information.”**

But the LLM answers:

> **“Anne Hathaway.”**

This is called a **hallucination**.
It mixed its training memory with the fresh context.

RAG fixes this by forcing the model to ONLY use the retrieved context.

---

2. Why not just give the model ALL your documents?

Many people think:

> “Models have huge 1M token context windows. Why not just stuff the whole company wiki into the prompt?”

Your teacher explains why this is a terrible idea:

Reason 1 — Cost

Processing 1M tokens is **hundreds of times** more expensive than processing a small 1K-token RAG prompt.

It becomes financially impossible.

Reason 2 — Latency (very slow)

A 1M-token prompt takes **30–60 seconds** to process BEFORE the model even answers.

For any real product → unacceptable.

Reason 3 — “Lost in the Middle” Problem

LLMs have a **U-shaped attention curve**:

- They remember the **start** of the prompt
- They remember the **end**
- They ignore the **middle**

Stuffing everything into one giant prompt makes things worse, not better.

Reason 4 — No Access Control

If you send ALL documents into the prompt:

- A user can indirectly see documents they should NOT have access to
- No permission system exists

- Everything is leaked to the model

RAG is your security layer
The retriever only fetches chunks the user is allowed to see.

⭐ Conclusion so far:

Large context ≠ good idea
RAG solves cost + speed + accuracy + security.

---

3. What is RAG (Retrieval-Augmented Generation)?

Your teacher's analogy:

> "RAG is an **open-book exam**."

When the model gets a question, it doesn't rely on memory.
Instead, it:

1. Goes to a **library** (Vector DB)
2. Finds the **right pages** (retrieval)
3. Reads ONLY those pages
4. Answer based strictly on them (generation)

This stops hallucinations and makes outputs grounded.

---

4. The RAG Pipeline (Step-by-Step)

Step 1 — User Query

Example:

> “What is our remote work policy?”

Step 2 — Query Embedding

Convert the query into a vector (numbers).

Step 3 — Vector Search

Search those numbers in the **Vector Database**, which returns the **Top-K most similar chunks**.

For example:

- Return the top 5 most relevant paragraphs.

Step 4 — Augment Prompt

Build a new prompt:

```
Context:
[chunk 1]
[chunk 2]
[chunk 3]
[chunk 4]
[chunk 5]

Query: What is our remote work policy?
Answer:
```

Step 5 — LLM Generation

The model ONLY uses these retrieved chunks to answer.

This is how hallucinations get reduced.

---

5. Why Chunking Is Needed (Ingestion Phase)

Before we can store anything in a Vector DB, we must **chunk** the text.

Why?

Imagine a **30,000-word PDF**.

Embedding that whole thing into **one giant vector** is impossible and useless.

We need a “Goldilocks size”:

- Not too big → inaccurate
- Not too small → loses meaning

So we break it into chunks.

---

6. Chunking Strategies

1. Fixed-Size Chunking

Split every 512 tokens.

Pros:

- Very simple

- Fast

Cons:

- Dumb — can cut sentences in half
- Meaning breaks

We add **overlap** (sliding window) to reduce cutting.

---

2. Content-Aware Chunking

Split where it makes sense:

- Paragraphs
- Headings (###)
- Sentences
- Punctuation (.)

Better than fixed size because it respects structure.

---

3. Recursive Chunking

The best approach.

It tries a series of rules:

1. Try to split by paragraph
2. If still too big → split by sentence
3. If still too big → split by space

Keeps chunks meaningful AND size-controlled.

---

7. Vector Database – What Is It?

Not a normal SQL database.

Think of it as a **library for numbers**.

It stores:

- Billions of **embeddings**
- Each vector = location of a text chunk in multidimensional space

It can:

- Find the **nearest neighbors** in milliseconds
  → meaning “find the most related paragraphs quickly”

This is what powers RAG.

---

8. RAG vs Fine-Tuning

Your teacher gives a perfect analogy:

Fine-Tuning = sending a student to a specialized school

- Changes the model’s brain (weights)
- Teaches new behaviors, tone, style, personality

RAG = giving the student an open-book in the exam

- Does NOT change the model’s brain
- Adds external knowledge

- Teaches factual content

Golden Rule from slides:

> **Use RAG to teach WHAT to say.**
> **Use fine-tuning to teach HOW to say it.**

Exactly this.

⭐ 9. Naive → Advanced (Fancy) RAG

Your teacher highlights two BIG problems:

Problem 1 — Retrieval Quality (Low Precision)

If your retriever returns irrelevant chunks:

- The model gets confused
- It might hallucinate
- Garbage in → garbage out

Problem 2 — Response Quality

Even if retrieval is correct, the LLM might still misinterpret the chunk
(because the chunk contains filler, distractions, or irrelevant text).

This is where advanced RAG tricks come in.

---

10. Pre-Retrieval Optimization — HyDE

HyDE = **Hypothetical Document Embeddings**

Steps:

1. Ask the LLM to hallucinate a possible answer
2. Use THAT hallucinated answer as the embedding
3. Retrieve based on that fake answer

Why it works:

- Questions are short and vague
- Fake answers are long and “dense”

Searching with a “fake answer” is more accurate than searching with a question.

---

11. Retrieval Optimization — Hybrid Search

Semantic search is good for meaning.
BUT it sucks at specific keywords like:

- product codes
- IDs
- exact names

Example:

- “ada-002”

So we combine:

Dense vector search (what it means)

Sparse keyword search (BM25) (what it literally says)

Together = **Hybrid Search**, which is the strongest.

---

12. Post-Retrieval Optimization — Re-Ranking

Hybrid search may return 50 possible documents.

We can't send all 50 to the LLM.

Solution:

- Use a **Cross-Encoder Re-Ranker**

Analogy from the slides:

- Retriever = "fast but dumb librarian, brings any book that mentions RAG"
- Re-ranker = "expert assistant reads all 50 books and gives each a 0-100 relevance score"

This ensures the BEST, MOST ACCURATE chunks are used.

---

13. Post-Retrieval Optimization — Prompt Compression

Even the best chunks contain:

- filler
- useless sentences

- irrelevant details

But the LLM should only see what matters.

So we use a **Prompt Compression model** like LLMLingua:

- Reads the retrieved chunks
- Removes unimportant tokens
- Outputs a compressed version with the same meaning
- Reduces context size
- Increases accuracy

This is one of the strongest improvements for advanced RAG.

⭐ Final Summary — The Full "RAG Stack"

Your teacher's lecture builds the following understanding:

RAG solves:

- Hallucinations
- High cost
- Slow latency
- Lost-in-the-middle
- Access control

The RAG Pipeline:

1. **Embedding** the query
2. **Retrieving** top-k chunks
3. **Augmenting** the prompt
4. **Generating** grounded answers

To improve RAG:

- HyDE (pre-retrieval)
- Hybrid Search (retrieval)
- Re-ranking (post-retrieval)
- Prompt Compression (post-retrieval)

These upgrades turn naive RAG into **advanced, production-level RAG**.

1. In the Shakespeare example, why does the model incorrectly answer “Anne Hathaway”?

A. The context was too long
B. Prior knowledge overrides missing context
C. The retriever returned irrelevant chunks
D. The model failed to embed the query

Answer: B

---

2. What is the *correct* grounded answer when the context does not contain the requested information?

A. The nearest plausible fact
B. The best guess
C. “I don’t know / Context does not contain that information.”
D. A hallucinated filler response

Answer: C

---

3. Why is simply using a 1M-token context window not a scalable solution?

A. Large prompts decrease accuracy
B. The model cannot embed long documents
C. It is extremely expensive and slow
D. It breaks positional encodings

Answer: C

---

4. What phenomenon explains why LLMs ignore information placed in the middle of very long prompts?

A. Cross-attention decay
B. Middle-token vanishing gradient
C. Lost-in-the-Middle effect
D. Token drift syndrome

Answer: C

---

5. Why is RAG considered a security layer?

A. It encrypts the prompt
B. It prevents the LLM from using embeddings
C. The retriever only returns chunks the user is allowed to access
D. It masks sensitive tokens inside the LLM

Answer: C

---

6. In RAG, what is the FIRST step after receiving a user query?

A. Retrieval
B. Chunk filtering
C. Query embedding
D. Prompt compression

Answer: C

---

7. What does the Vector Database store?

A. Full documents
B. Text files indexed by BM25
C. High-dimensional embeddings of text chunks
D. Tokenized sequences

Answer: C

---

8. Why do we need to split documents into chunks?

A. Embedding models cannot embed raw text
B. Large documents cause model hallucinations
C. Vector search only supports fixed-length input
D. Embedding entire documents at once is impossible and imprecise

Answer: D

---

9. What is the risk of using fixed-size chunking without overlap?

A. Increased retrieval latency
B. Chunks may cut sentences and lose meaning
C. Embeddings become too small
D. The vector DB cannot store them

Answer: B

---

10. Content-aware chunking attempts to split using:

A. Random token lengths
B. Only periods
C. Natural text boundaries
D. Character frequency

Answer: C

---

11. Recursive chunking splits using:

A. First characters, then symbols
B. A sequence of preferred separators until the size fits
C. A fixed 512-token window
D. Semantic similarity

Answer: B

---

12. The main job of Hybrid Search is to:

A. Compress retrieved chunks
B. Combine semantic search with keyword search
C. Replace vector search with BM25
D. Reduce storage cost

Answer: B

---

13. Dense semantic search is best for:

A. Product IDs
B. Exact term matching
C. Abstract or conceptual questions
D. Keyword-heavy queries

Answer: C

---

14. Sparse/BM25 search is best for:

A. Concepts like “leadership strategy”
B. Long passages
C. Rare entities or product codes
D. Summarization tasks

Answer: C

---

15. What happens after Hybrid Search returns a large number of candidate documents?

A. They are all sent to the LLM
B. They are concatenated into the final prompt
C. A Cross-Encoder re-ranks them
D. They are discarded

Answer: C

---

16. Why is re-ranking needed?

A. Because BM25 is slow
B. Vector databases cannot sort results
C. Stage-1 retrieval is fast but imprecise
D. LLMs require sorted chunks

Answer: C

---

17. The Cross-Encoder re-ranker computes:

A. Independent scores for documents
B. Query–document pair relevance scores
C. Cosine similarity only
D. BM25-weighted scores

Answer: B

---

18. The goal of prompt compression is to:

A. Increase token count so the model reads more
B. Condense chunks by removing irrelevant tokens
C. Generate summaries
D. Reduce cost by lowering LLM temperature

Answer: B

---

19. In RAG, what is the model *not* allowed to do?

A. Retrieve new knowledge itself
B. Embed documents
C. Answer using outside prior knowledge
D. Use self-attention

Answer: C

---

20. Why is RAG considered an “open-book exam”?

A. The model reads the entire corpus every time
B. The model memorizes everything
C. It retrieves only the relevant “pages”
D. It increases hallucinations

Answer: C

---

21. Which of the following best describes “HyDE”?

A. A chunk summarization technique
B. A hallucinated fake-answer generator for better retrieval
C. A re-ranking model
D. A vector database algorithm

Answer: B

---

22. Why does HyDE improve retrieval accuracy?

A. It increases chunk size
B. It increases BM25 weight
C. Fake answers are more semantically dense than questions
D. It bypasses embedding models

Answer: C

---

23. RAG vs Fine-Tuning — which statement is correct?

A. RAG changes model weights, fine-tuning does not
B. Fine-tuning teaches knowledge, RAG teaches style
C. RAG is inference-time; fine-tuning is training-time
D. Both modify how the model stores facts internally

Answer: C

---

24. According to the slides, what should Fine-Tuning be used for?

A. Adding factual knowledge
B. Changing the personality or tone
C. Expanding context windows
D. Improving retrieval accuracy

Answer: B

---

25. How does RAG reduce hallucinations?

A. It increases embedding vector size
B. It forces the model to only use retrieved context
C. It reduces number of heads in attention
D. It finetunes the model

Answer: B

---

26. The purpose of overlap in fixed-size chunking is to:

A. Reduce embedding cost
B. Prevent semantic cutting
C. Improve BM25 ranking
D. Remove irrelevant sentences

Answer: B

---

27. What is the role of the “Top-k” in vector search?

A. Select the most diverse chunks
B. Select the k most similar chunk embeddings
C. Sort by metadata
D. Retrieve all chunks in the database

Answer: B

---

28. Prompt compression models like LLMLingua operate:

A. Before retrieval
B. After retrieval, before LLM generation
C. After generation
D. Inside the vector database

Answer: B

---

29. The biggest disadvantage of using very large context windows instead of RAG is:

A. Lower accuracy
B. No GPU support

C. Extremely high cost and slow latency
D. It doesn’t allow tokenization

Answer: C

---

30. A RAG system fails if:

A. The model’s temperature is too low
B. Retrieval returns irrelevant or noisy chunks
C. Positional encodings are turned off
D. BM25 is not installed

Answer: B